ADR-003: エージェントオーケストレーション — LangGraph を採用

ステータス

承認

日付

2026-03-12

コンテキスト

シャチョツーの **BPO Worker**（実行エージェント）は、外部SaaS（kintone・freee・Slack等）を自律的に操作して業務を代行する。この実行フローには以下の複雑な要件がある。

BPO Worker の要件

要件	詳細
ステートフルなマルチステップ実行	「顧客データ取得 → 請求書作成 → freeeへの登録 → Slackで社長に報告」のような多段階タスク
Human-in-the-Loop (HitL)	重要な操作（例: 外部への送金・メール送信）前に人間（社長）の承認を取る
条件分岐	中間結果によって次のステップが変わる（例: 取得データが空なら別ルート）
状態の永続化	セッションを跨いだ実行継続（例: 承認待ち中にセッションが切れても再開できる）
エラーリカバリ	ステップ失敗時にリトライ・代替ルートへの切り替え
可観測性	実行ステップをログ・UI で可視化できる
並列実行	独立したサブタスクを並列で実行して高速化

設計上の哲学

BPO Workerは「社長の代理人」として行動する。誤操作・誤送信のリスクを最小化するため、**高リスク操作には必ず人間の承認ゲートを設ける**設計方針を取る。これはLLMエージェントの自律性とトレードオフになるが、中小企業の経営者が信頼して使うためには必須の設計判断である。

検討した選択肢

選択肢A: LangGraph — 採用案

概要: LangChainが提供するグラフベースのエージェントオーケストレーションフレームワーク。ノード（処理単位）とエッジ（遷移条件）でワークフローを定義する。

メリット:

1. **HitL ネイティブサポート**: `interrupt_before` / `interrupt_after` APIにより、任意のノード前後で実行を一時停止し、人間の入力を待つことができる。外部サービスとの連携なしに実装可能

```python
from langgraph.graph import StateGraph

builder = StateGraph(BPOState)
builder.add_node("plan", plan_node)
builder.add_node("execute", execute_node)
builder.add_node("confirm", confirm_node) # 社長への確認

# "confirm" ノードの前で一時停止し、社長の入力を待つ
graph = builder.compile(
checkpointer=checkpointer,
interrupt_before=["execute"] # 実行前に必ず承認
)
```

2. **永続化チェックポインター**: `PostgresSaver` によりSupabaseのPostgreSQLにグラフ状態を保存。セッション跨ぎの実行継続が可能

```python
from langgraph.checkpoint.postgres import PostgresSaver

checkpointer = PostgresSaver.from_conn_string(
settings.SUPABASE_CONNECTION_STRING
)
```

graph = builder.compile(checkpointer=checkpointer)
```

3. **グラフベースの条件分岐**: エッジに条件関数を設定することで、中間結果に応じた動的ルーティングが自然に表現できる

```python
def route_by_result(state: BPOState) -> str:
if state["data_found"]:
return "process"
else:
return "notify_no_data"

builder.add_conditional_edges("fetch", route_by_result)
```

4. **並列実行**: `Send` APIでサブグラフを並列実行できる。独立した複数タスクの同時実行が可能
5. **Python ネイティブ**: FastAPIと同一スタック。LangChainのツール・メモリ抽象化とシームレスに統合できる
6. **LangSmith統合**: 実行トレース・デバッグ・コスト監視が可能。開発効率と可観測性を両立
7. **Streaming対応**: SSE（Server-Sent Events）でノードの中間出力をリアルタイムにフロントエンドへ配信可能

デメリット:

- **学習曲線**: グラフ構造（ノード/エッジ/状態管理）の概念理解が必要。LangChainの他APIとは異なる設計パラダイム
- **バージョン安定性**: LangGraphは比較的新しいライブラリで、マイナーバージョンアップで破壊的変更が発生することがある
- **デバッグ難度**: グラフの実行フローが複雑になるとデバッグが難しくなる（LangSmithで緩和）
- **オーバーヘッド**: シンプルな単発タスク（LLM1回呼び出しで完結するもの）にはオーバースペック

選択肢B: CrewAI — 却下

概要: 「エージェントチーム」という概念でマルチエージェント協調を実現するフレームワーク。

メリット:
- ロールベースのエージェント定義が直感的（「リサーチャー」「ライター」等）
- 高レベルの抽象化で素早くエージェントを構築できる
- コミュニティが活発
```

デメリット:
- **状態制御の粒度**: CrewAIは「チームに仕事を投げる」高レベルAPIで、ステップごとの細かい状態管理・条件分岐の制御が難しい
- **HitL 非標準**: 途中で人間の承認を挟む機能が標準でない。カスタム実装が必要
- **永続化**: チェックポインター機能が弱い。セッション跨ぎの実行継続に別途実装が必要
- **ブラックボックス**: 内部の実行フローが見えにくく、デバッグが困難
- **コスト制御**: 各エージェントが何回LLMを呼び出すかが予測しにくい。コスト爆発リスク

→ **決定**: HitLと状態管理の要件を満たせない。却下。

選択肢C: Microsoft AutoGen — 却下

概要: Microsoftが提供するマルチエージェント会話フレームワーク。エージェント間の対話でタスクを解決する。

メリット:
- マルチエージェント会話が柔軟
- コード実行サンドボックスが組み込み
- GPT-4との親和性が高い

デメリット:
- **Microsoft依存**: Microsoftのエコシステムへの依存。Azure OpenAI Service利用が前提になりやすい
- **Python非標準**: 日本語ドキュメントが少ない
- **HitL 設計**: 会話ループで人間を1エージェントとして扱う設計。承認フローの実装が煩雑
- **状態永続化**: 本番環境での状態永続化が標準で難しい
- **GCPとの統合**: Cloud RunやSupabaseとの統合実績が少ない

→ **決定**: Microsoftエコシステムへの依存とHitL実装の複雑さが問題。却下。

選択肢D: カスタムステートマシン (Python) — 却下

概要: Pythonの `asyncio` や `dataclasses` を使って独自にステートマシンを実装する。

メリット:
- 完全なコントロール
- 外部依存なし
- プロジェクト固有の最適化が可能

デメリット:
- **実装コスト**: チェックポインター・HitL・条件分岐・並列実行・エラーリカバリを全て自前で実装する必要がある。推定3〜4週間

- **テスト負担**: フレームワーク自体のバグ修正・テストも自社負担
- **拡張性**: 要件変更のたびにステートマシンの設計見直しが必要
- **可観測性**: 実行トレーシング・デバッグツールも自前実装

→ **決定**: 1人開発チームには実装・維持コストが過大。LangGraphで既存実装を活用すべき。却下。

選択肢E: n8n / Zapier (iPaaS) — 却下

概要: ノーコードのワークフロー自動化ツール。

メリット:
- UIで直感的にフロー設計できる
- 300以上のSaaS連携が標準搭載

デメリット:
- **LLM統合**: カスタムLLMロジック（状態依存の判断、プロンプト動的生成）の実装が困難
- **コード制御**: ビジネスロジックをコードで管理できない。Git管理不可
- **コスト**: Zapierの本番プランは高額。n8nセルフホストは運用負担
- **プロダクト価値**: シャチョツー自体がワークフロー自動化製品。競合ツールを内部で使うのはプロダクト価値の自己否定

→ **決定**: プロダクトの差別化要因を外部ツールに依存することは戦略的に不可。却下。

決定

LangGraph を BPO Worker のエージェントオーケストレーションフレームワークとして採用する。

合わせて以下を決定する:

- **チェックポインター**: `PostgresSaver`（Supabase PostgreSQL）
- LangGraphのグラフ状態をSupabaseのテーブルに保存
- `company_id` をスレッドIDのプレフィックスとして使用し、テナント分離を維持
- **HitL トリガー**: 以下の操作タイプは `interrupt_before` を必須とする
- `WRITE`（外部SaaSへのデータ書き込み）
- `SEND`（メール・Slack・外部通知の送信）
- `PAYMENT`（決済・振込・請求書発行）
- `DELETE`（データ削除操作）
- **LangSmith**: 開発・ステージング環境でトレーシングを有効化。本番はオプトイン

グラフ設計パターン

BPO Workerの標準グラフ構造:

```
START
  ↓
[plan_node]          ← タスクを実行計画に分解
  ↓
[safety_check_node] ← リスク評価・HitL判定
  ↓（高リスク操作）
[human_approval]     ← interrupt_before: ここで一時停止
  ↓（承認後 or 低リスク）
[execute_node]       ← SaaS操作実行
  ↓
[verify_node]        ← 実行結果確認
  ↓
[report_node]        ← 社長へのサマリー報告
  ↓
END
```

影響

ポジティブな影響

- **開発速度**: HitL・チェックポインター・並列実行を自前実装せずに実現。推定2〜3週間の開発工数削減
- **信頼性**: `PostgresSaver` によりサーバー再起動・障害後も状態から再開可能
- **透明性**: LangSmithで全ステップの入出力・コスト・レイテンシを可視化。バグ調査が容易
- **将来拡張**: Engineer Worker（要件定義対話）・Brain推論フローにも同じパターンを適用できる

負の影響・トレードオフ

- **LangGraph学習コスト**: グラフ設計パターン（ノード/エッジ/状態スキーマ）の習得に1週間程度を見込む
- 緩和策: LangGraphの公式チュートリアルをStep 0中に完了する
- **バージョン管理**: LangGraphのマイナーバージョンアップで破壊的変更が発生する可能性
- 緩和策: `requirements.txt` でバージョンをピン留めし、アップグレードはCHANGELOGを確認してから実施
- **シンプルタスクへのオーバーヘッド**: 1ステップで完結するタスクにもグラフ構造が必要
- 緩和策: 単純なLLM呼び出しは `llm/client.py` 直接呼び出しで済ませ、LangGraphは複数ステップのワークフロー専用とする

今後の制約

1. **BPO Workerは全てLangGraph経由**: 直接SaaS APIを叩くコードをルーターに書かない
2. **スレッドIDフォーマット**: `{company_id}:{task_id}` とし、company_id プレフィックスでテナント分離を保証
3. **状態スキーマはPydanticで定義**: `TypedDict` ではなく `BaseModel` を使用し、型安全性を確保
4. **HitLのタイムアウト**: 承認待ちのタスクは48時間でタイムアウト。社長への再通知を1回送信してからキャンセル

関連

- `shachotwo/02_プロダクト設計.md` — Section 4: エージェント設計
- `workers/bpo/` — BPO Worker 実装
- `workers/engineer/` — Engineer Worker 実装（同パターン適用）
- ADR-001: Gemini LLM採用（LangGraphのノード内でLLMを呼び出す）
- ADR-002: Supabase採用（PostgresSaverの格納先）